

```
1: #include <stdio.h>
2: #define MAX 20
3:
4: // Estructura que representa un poste (palo)
5: typedef struct {
6:     int discos[MAX]; // arreglo de discos (base en índice 0, tope en el último
ocupado)
7:     int tope;         // índice del último elemento (-1 si está vacío)
8: } Poste;
9:
10: // Inicializar un poste vacío
11: void inicializar_poste(Poste *p) {
12:     p->tope = -1;
13: }
14:
15: // Verificar si un poste está vacío
16: int esta_vacio(Poste *p) {
17:     return p->tope == -1;
18: }
19:
20: // Verificar si un poste está lleno
21: int esta_lleno(Poste *p) {
22:     return p->tope == MAX - 1;
23: }
24:
25: // Obtener el disco en el tope (sin quitarlo)
26: int ver_tope(Poste *p) {
27:     if (esta_vacio(p)) return -1;
28:     return p->discos[p->tope];
29: }
30:
31: // Apilar un disco en el poste (lo pone en el tope)
32: void apilar(Poste *p, int disco) {
33:     if (esta_lleno(p)) {
34:         printf("Error: poste lleno\n");
35:         return;
36:     }
37:     p->tope++;
38:     p->discos[p->tope] = disco;
39: }
40:
41: // Desapilar el disco del tope y retornarlo
42: int desapilar(Poste *p) {
43:     if (esta_vacio(p)) {
44:         printf("Error: poste vacío\n");
45:         return -1;
46:     }
47:     int disco = p->discos[p->tope];
48:     p->tope--;
49:     return disco;
50: }
51:
52: // Mover un disco desde un poste origen a un poste destino
53: // Retorna 1 si el movimiento es válido, 0 si no
54: int mover_disco(Poste *origen, Poste *destino, char nombre_origen, char nombre_des
tino) {
55:     if (esta_vacio(origen)) {
56:         printf("Movimiento invalido: poste %c esta vacío\n", nombre_origen);
57:         return 0;
58:     }
59:
60:     int disco_origen = ver_tope(origen);
61:     int disco_destino = ver_tope(destino);
62:
63:     if (!esta_vacio(destino) && disco_origen > disco_destino) {
64:         printf("Movimiento invalido: no se puede poner disco %d sobre disco %d\n",
65:             disco_origen, disco_destino);
66:         return 0;
67:     }
}
```

```
68:
69:     // Realizar el movimiento
70:     int disco = desapilar(origen);
71:     apilar(destino, disco);
72:     printf("Mover disco %d de %c a %c\n", disco, nombre_origen, nombre_destino);
73:     return 1;
74: }
75:
76: // Funci3n recursiva de las Torres de Hanoi
77: // n: n3mero de discos a mover
78: // origen: poste de donde se mueven
79: // destino: poste a donde se mueven
80: // auxiliar: poste auxiliar
81: // nom_o, nom_d, nom_a: nombres para imprimir (ej: 'A', 'B', 'C')
82: void hanoi(int n, Poste *origen, Poste *destino, Poste *auxiliar,
83:            char nom_o, char nom_d, char nom_a) {
84:     // CASO BASE: mover un solo disco
85:     if (n == 1) {
86:         mover_disco(origen, destino, nom_o, nom_d);
87:         return;
88:     }
89:
90:     // PASO RECURSIVO
91:     // 1. Mover n-1 discos de origen a auxiliar (usando destino como apoyo)
92:     hanoi(n - 1, origen, auxiliar, destino, nom_o, nom_a, nom_d);
93:
94:     // 2. Mover el disco m3s grande de origen a destino
95:     mover_disco(origen, destino, nom_o, nom_d);
96:
97:     // 3. Mover n-1 discos de auxiliar a destino (usando origen como apoyo)
98:     hanoi(n - 1, auxiliar, destino, origen, nom_a, nom_d, nom_o);
99: }
100:
101: // Funci3n para mostrar el estado actual de los tres postes
102: void mostrar_estado(Poste *A, Poste *B, Poste *C, char nombreA, char nombreB, char
nombreC) {
103:     printf("\n--- Estado actual ---\n");
104:
105:     printf("Poste %c: ", nombreA);
106:     if (esta_vacio(A)) printf("vacio");
107:     else {
108:         for (int i = 0; i <= A->tope; i++) {
109:             printf("%d ", A->discos[i]);
110:         }
111:     }
112:     printf("\n");
113:
114:     printf("Poste %c: ", nombreB);
115:     if (esta_vacio(B)) printf("vacio");
116:     else {
117:         for (int i = 0; i <= B->tope; i++) {
118:             printf("%d ", B->discos[i]);
119:         }
120:     }
121:     printf("\n");
122:
123:     printf("Poste %c: ", nombreC);
124:     if (esta_vacio(C)) printf("vacio");
125:     else {
126:         for (int i = 0; i <= C->tope; i++) {
127:             printf("%d ", C->discos[i]);
128:         }
129:     }
130:     printf("\n\n");
131: }
132:
133: int main() {
134:     Poste A, B, C;
135:     int n;
136:
```

```
137: // Inicializar los tres postes
138: inicializar_poste(&A);
139: inicializar_poste(&B);
140: inicializar_poste(&C);
141:
142: // Pedir número de discos
143: printf("Ingrese el numero de discos: ");
144: scanf("%d", &n);
145:
146: // Inicializar el poste A con los discos (el más grande abajo)
147: // Para que [1,2,3,4,5] funcione, asumimos que 1 es el más pequeño, n es el
más grande
148: // Los apilamos de manera que el más grande quede abajo (índice 0) y el más
pequeño arriba (tope)
149: printf("Inicializando poste A con discos del 1 (pequeno) al %d (grande)\n", n)
;
150: for (int i = n; i >= 1; i--) {
151:     apilar(&A, i);
152: }
153:
154: printf("\nEstado inicial:\n");
155: mostrar_estado(&A, &B, &C, 'A', 'B', 'C');
156:
157: printf("\nResolviendo Torres de Hanoi para %d discos:\n", n);
158: hanoi(n, &A, &B, &C, 'A', 'C', 'B');
159:
160: printf("\nEstado final:\n");
161: mostrar_estado(&A, &B, &C, 'A', 'B', 'C');
162:
163: return 0;
164: }
```